

Implémentation, ServeurMultiThreade (v3)

Fabrice.Kordon@lip6.fr



Objectif de cette révision

2

Remplacer la file de messages

- Se baser sur `ArrayBlockingQueue<E>`
- Remplacer `insérerRequete()` par `put()`
 - ▶ **bloquant**
- Remplacer `extraireRequete()` par `take()`
 - ▶ **bloquant aussi**

Les «utilitaires»

3

```
import java.util.Random;  
//import java.util.concurrent.locks.*;  
import java.util.concurrent.ArrayBlockingQueue;
```

```
class Requete {  
    UnClient emetteur;  
    int valeur;  
  
    Requete(UnClient c, int val) {  
        valeur = val;  
        emetteur = c;  
    }
```

```
    public String toString () {  
        return "(" + emetteur + ", " + valeur + ")" ;  
    }  
}
```

Plus besoin de
ReentrantLock!



Et plus de classe FileRequetes
non plus (72 LoC)



La classe LeServeur

4

```
class LeServeur implements Runnable {
    private ArrayBlockingQueue<Requete> f;

    LeServeur (ArrayBlockingQueue<Requete> fr) {
        f = fr;
    }

    private void trace (String m) {
        System.out.println ("Serveur : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        try {
            Requete r;
            this.trace("initialisé");
            while (true) {
                r = f.take(); // potentiellement bloquant
                this.trace("je traite " + r);
                new Thread (new Servant(r)).start();
            }
        } catch (Exception e) { // Gérer la terminaison
            this.trace("il n'y a plus personne, adieu monde cruel");
        }
    }
}
```

La classe unClient

5

```
class unClient implements Runnable {
    private int monId;
    private ArrayBlockingQueue<Requete> f;

    private Random generateur = new Random();
    private final Object reveil = new Object(); // attente réponse

    private static int cpt = 1;
    private static final Object mutex = new Object();
    private static int clientsActifs = 0; // compter les clients
    private Thread serveur; // connaître le serveur

    unClient (ArrayBlockingQueue<Requete> fr, Thread serv) {
        f = fr;
        serveur = serv;
        synchronized (mutex) {
            monId = cpt++;
            clientsActifs++;
        }
    }

    private void trace (String m) {...}

    public void requeteTraitee () {...}

    public String toString () {...}
}
```

La classe unClient

5

```
class UnClient implements Runnable {
    private int monId;
    private ArrayBlockingQueue<Requete> f;

    public void run() {
        this.trace("initialisé");
        Requete r = new Requete(this, generateur.nextInt(1000));
        this.trace("j'envoie la requête " + r);
        try {
            f.put(r);
            this.trace("j'attends le serveur");
            synchronized (reveil) {
                reveil.wait();
            }
        }
        catch (InterruptedException e) {
            System.out.println ("UnClient "+monId+", cela doit jamais arriver, " + e);
        }
        this.trace("ma requête a été traitée");
        synchronized(mutex) {
            clientsActifs--;
            if (clientsActifs == 0) {
                this.trace("étant le dernier, je tue le serveur");
                serveur.interrupt();
            }
        }
    }
}
```

La «classe principale»

6

```
public class ServeurMultiThreade3 {  
  
    static int tailleFile;  
    static int nbClients;  
  
    public static void main(String []args) {  
        try {  
            tailleFile = Integer.parseInt(args[0]);  
            nbClients = Integer.parseInt(args[1]);  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.err.println ("Arguments requis, taille de la file, nombre de clients.");  
            return;  
        }  
        // Lancer le système  
        ArrayBlockingQueue<Requete> laFile =  
            new ArrayBlockingQueue<Requete>(tailleFile, true);  
        Thread s = new Thread(new LeServeur(laFile));  
        for (int i = 0; i < nbClients ; i++) {  
            new Thread(new UnClient(laFile, s)).start();  
        }  
        s.start();  
    }  
}
```

Et la classe Servant?

7

Aucun changement

- 🔔 N'accède pas à la file de messages



Une exécution

8

```
$ java ServeurMultiThreade3 2 3
UnClient 1 : initialisé
Serveur : initialisé
UnClient 2 : initialisé
UnClient 3 : initialisé
UnClient 1 : j'envoie la requête (client 1, 254)
UnClient 3 : j'envoie la requête (client 3, 361)
UnClient 2 : j'envoie la requête (client 2, 911)
UnClient 1 : j'attends le serveur
Serveur : je traite (client 1, 254)
UnClient 2 : j'attends le serveur
UnClient 3 : j'attends le serveur
Serveur : je traite (client 3, 361)
Servant (client 1, 254) : initialisé
Serveur : je traite (client 2, 911)
Servant (client 3, 361) : initialisé
Servant (client 2, 911) : initialisé
Servant (client 1, 254) : je prévien le client
UnClient 1 : ma requête a été traitée
Servant (client 3, 361) : je prévien le client
UnClient 3 : ma requête a été traitée
Servant (client 2, 911) : je prévien le client
UnClient 2 : ma requête a été traitée
UnClient 2 : étant le dernier, je tue le serveur
Serveur : il n'y a plus personne, adieu monde cruel
$
```

En guise de conclusion...

9

Solution intéressante...

- Mais liée à Java
 - ▶ Riche d'un ensemble d'utilitaires



Mais... dans la vraie vie

- Ces utilitaires peuvent ne pas exister!
 - ▶ Il faut les connaître
- Il est important de bien comprendre les framework
 - ▶ Et donc les maîtriser...

